gRPC
R Project for Statistical Computing
GSoC 2020

Google Summer of Code

Shehab Hosny Ibrahim
Faculty of Engineering Ain Shams University
Cairo, Egypt
Computer Engineering

March 2020

Table of Contents

1 About Me 2
  1.1 Contact Information 2
  1.2 Affiliations 2

2 Mentor Details. 2

3 About the Project 3
  3.1 About gRPC. 3
  3.2 Features to be added to R library for gRPC in this Project 3

4 Implementation - Coding Plans and Methods 4
  4.1 Technical Implementation. 5
  4.2 Testing and Validation on various test case scenarios 11
  4.3 Make gRPC package ready to be avilable on CRAN. 11
  4.4 Creating the Reference Manual. 13
  4.5 Development Environment. 13

5 Timeline 14

6 Tests Solution 17
  6.1 Easy Test 17
  6.2 Medium Test 19
  6.3 Hard Test 21

7 Additional Commitments 22
8 Why This Project? 22
9 Why Me? 22

1. About Me

1.1. Contact Information

- Name
- Country
- Email
- LinkedIn
- Github

1.2. Affiliations

- University Ain Shams
- Faculty Engineering
- Program Computer Engineering

2. Mentor Details

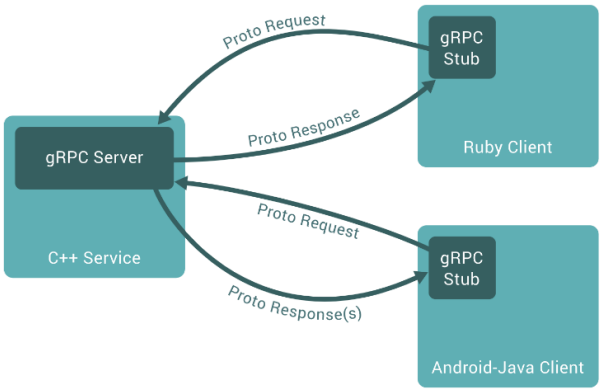
EVALUATING MENTOR
Email Address

SECOND MENTOR
Email Address

3. About the Project

Project Page URL → https://github.com/rstats-gsoc/gsoc2020/wiki/grpc

3.1. About gRPC



gRPC represents Protocol Buffers running over HTTP/2 with multiple programming language support. In gRPC, a client application can directly call a method on a server application on a different machine as if it were a local object, making it easier to create distributed applications and services. Generally, gRPC is based around the idea of defining a service, specifying the methods that can be called remotely with their parameters and return types. On the server side, the server implements this interface and runs a gRPC server to handle client calls. On the client side, the client has a stub that provides the same methods as the server.

3.2. Features to be added to R library for gRPC in this Project

- Authentication:
  The ability to use Security Tokens , as a metadata attached to each message, to allow only authenticated users to access protected resources.

- Security:
  The ability to use TLS (Transport Layer Security) to encrypt the messages sent and latter on decrypt the encrypted messages received. This needs to take place as soon as a new connected is initiated.

- Streaming:
  The ability to send Multiple Messages over a Single Connection. Streaming is useful in any scenario where the server needs to send multiple items in response to a client call. This may be used to reduce the number of requests required to gather the data, instead, bundling them into a single response.

4. Implementation - Coding Plans and Methods

To ensure a smooth workflow of the project, the project objectives need to be broken down into smaller tasks where each task is accomplished periodically so that after finishing each task there will be sufficient time for both documentation and testing.

Main Objectives of the Project:
The aim of this year's GSoC is to add three more features to the gRPC package. The R wrapper to the C libraray was coded Neal Fultz back in 2017, but it lacks three main features: Authentication, Security and Streaming. These features will latter allow R to interact better with other gRPC projects, such as Apache now and many other Google machine learning products.

This project can be broken down into the following tasks:

- Implementing a distinct function (or set of functions) for each of the following:
  - Authenticating Messages
  - Securing Messages
  - Streaming Messages in a single connection

- Testing and Validating the Code on various test case scenarios.

- Make gRPC package ready to be avilable on CRAN
  - Documentation of each and every function
  - Writing tests for the code
  - Closing the open issues
  - Make sure the package passes all build tests
  - Cleanup of code before pushing it to Github

- Creating a Reference Manual.

I've extensively studied the gRPC package, both the R and C++ packages, and then made this proposal accordingly. Any changes to improve it are welcome. I believe, I will be ready before the community bonding period to attempt this task. According to my preparations, I've planned the following Technical Implementation to accomplish the task of the main project and implement the required functions (or set of functions) for Authentication, Security and Streaming.

4.1. Technical Implementation

Implementing a function to encrypt the messages sent using TLS

My approach is to modify the underlying C wrapper code for both the clients/senders and servers/receivers and implement the required functions . This will help me to use of some of the impelemented functions in the C++ gRPC package and map them to the underlying C wrapper code. I will then reflect these edits in the R code for both the clients and servers. My approach can be broken down into smaller steps as follows:

1) Recalling the C++ gRPC Package we can make use of the following Server Security and Authentication Functions:

SslCredentials()

```
static inline std::shared_ptr<ChannelCredentials> SslCredentials(
    const SslCredentialsOptions &options) {
  return ::grpc_impl::SslCredentials(options);
}
```

SslCredentialsOptions

```
struct SslCredentialsOptions
{
  //PEM encoding Buffer of the server root certificates
  grpc::string pem_root_certs;

  //PEM encoding Buffer of the client's private key
  grpc::string pem_private_key;

  //PEM encoding Buffer of the client's certificate chain
  grpc::string pem_cert_chain;
};
```

Creating server authentication with SSL/TLS

```
auto channel_creds = grpc::SslCredentials(grpc::SslCredentialsOptions());
auto channel = grpc::CreateChannel("message", channel_creds);
std::unique_ptr<Greeter::Stub> stub(Greeter::NewStub(channel));
```

2) Code snippets in the underlying C wrapper which will be modified:

Let's consider the function TLS_Secure_Channel() to be the newly implemented function to create a secured (encrypted) channel using TLS in the underlying C wrapper code.

TLS_Secure_Channel() function prototype
[To be implemented using the pre-defined function in the C++ package]

```
grpc_channel* TLS_Secure_Channel(const char* target,
                                 const grpc_channel_args* args,
                                 void* reserved);
```

Create a Secure Channel S_channel in fetch() function in client.cpp

```
grpc_channel *S_channel = TLS_Secure_Channel(server[0], NULL, RESERVED);
```

Involove the Secure Channel in the connection

```
c = grpc_channel_create_call(
    S_channel, NULL, GRPC_PROPAGATE_DEFAULTS, cq,
    method_slice, //grpc_slice_from_static_string("/foo"),
    sp,
    deadline,
    NULL);
```

3) Regarding the R code of client.r nothing will be changed to the point as the fetch() function used in the Rcpp function was already modified in the cpp file.

Code snippet used in the R file:

```
list(
          call = function(x, metadata=character(0))
          read( ResponseDescriptor,
                fetch(channel, fn$name, serialize(x, NULL), metadata)
              ),
          build = function(...) new(RequestDescriptor, ...)
     )
```

Implementing a function to send Authentication Tokens

I will follow here the same approach of the previous implementation to modify and implement new functions required for authentication.

1) Recalling the C++ gRPC Package we can make use of the following Server Security and Authentication Functions:

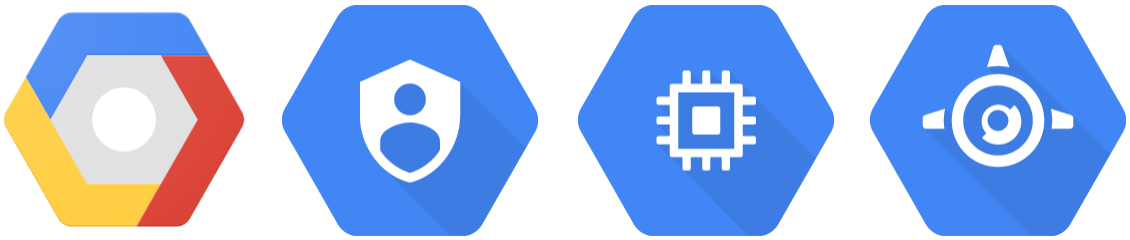
GoogleDefaultCredentials()

```
static inline std::shared_ptr<grpc_impl::ChannelCredentials>
GoogleDefaultCredentials() {
return ::grpc_impl::GoogleDefaultCredentials();
}
```

Authenticate the connection with Google

```
auto creds = grpc::GoogleDefaultCredentials();
auto channel = grpc::CreateChannel("greeter.googleapis.com", creds);
std::unique_ptr<Greeter::Stub> stub(Greeter::NewStub(channel));
```

Background about Google Credentials to Authenticate the connection:



Google Cloud client libraries use a strategy called Application Default Credentials (ADC) to find your application's credentials. When your code uses a client library, the strategy checks for your credentials in the following order:

1) First, ADC checks to see if the environment variable GOOGLE_APPLICATION_CREDENTIALS is set. If the variable is set, ADC uses the service account file that the variable points to.

2) If the environment variable isn't set, ADC uses the default service account that Compute Engine, Google Kubernetes Engine, Cloud Run, App Engine, and Cloud Functions provide, for applications that run on those services.

3) If ADC can't use either of the above credentials, an error occurs.

2) Code snippets in the underlying C wrapper which will be modified:

Let's consider the function Authenticate_Channel() to be the newly implemented function to send authentication tokens using Google Credentials in the underlying C wrapper code.

Authenticate_Channel() function prototype
[To be implemented using Google Credentials]

```
grpc_channel* Authenticate_Channel(grpc_channel* target);
```

Create an Authenticate Channel A_channel in fetch() function in client.cpp
This can be done after encrypting the channel as follows:

```
grpc_channel *S_channel = TLS_Secure_Channel(server[0], NULL, RESERVED);
grpc_channel *A_channel = Authenticate_Channel(S_channel);
```

Involove the Authenticated Channel in the connection

```
c = grpc_channel_create_call(
    A_channel, NULL, GRPC_PROPAGATE_DEFAULTS, cq,
    method_slice, //grpc_slice_from_static_string("/foo"),
    sp,
    deadline,
    NULL);
```

3) Regarding the R code of client.r nothing will be changed to the point as the fetch() function used in the Rcpp function was already modified in the cpp file.

Code snippet used in the R file:

```
list(
          call = function(x, metadata=character(0))
          read( ResponseDescriptor,
                fetch(channel, fn$name, serialize(x, NULL), metadata)
              ),
          build = function(...) new(RequestDescriptor, ...)
     )
```

Implementing a function to allow Streaming over a single connection

There are three main types for streaming:

1) Server-side streaming:
   The client sends a request to the server and gets a stream to read a sequence of messages back. The client reads from the returned stream until there are no more messages.

2) Client-side streaming:
   The client writes a sequence of messages and sends them to the server, again using a provided stream. Once the client has finished writing the messages, it waits for the server to read them and return its response.

3) Bidirectional streaming
   Both sides send a sequence of messages using a read-write stream. The two streams operate independently, so clients and servers can read and write in whatever order they like.

I will follow here the same approach of the previous implementation to modify and implement new functions required for authentication.

1) Recalling the C++ gRPC Package we can make use of the following Server Streaming Functions:

   ListFeatures() → Server-side streaming RPC

```
Status ListFeatures(ServerContext* context, const Rectangle* rectangle,
                    ServerWriter<Feature>* writer) override {
  auto lo = rectangle->lo();
  auto hi = rectangle->hi();
  long left = std::min(lo.longitude(), hi.longitude());
  long right = std::max(lo.longitude(), hi.longitude());
  long top = std::max(lo.latitude(), hi.latitude());
  long bottom = std::min(lo.latitude(), hi.latitude());
  for (const Feature& f : feature_list_) {
    if (f.location().longitude() >= left &&
        f.location().longitude() <= right &&
        f.location().latitude() >= bottom &&
        f.location().latitude() <= top) {
      writer->Write(f);
    }
  }
  return Status::OK;
}
```

RecordRoute() → Client-side streaming RPC

This function is similar to ListFeatures(). However, the only difference is that we get a Server reader instead of a request object and a single response. We use the Read() method to repeatedly read in our client's requests to a request object until there are no more messages. The following code snippet represents the differnce between the two functions.

```
while (stream -> Read(&point)) {
  ...//process client input
}
```

RouteChat() → Bidirectional streaming RPC

```
Status RouteChat(ServerContext* context,
                 ServerReaderWriter<RouteNote, RouteNote>* stream)
override {
  std::vector<RouteNote> received_notes;
  RouteNote note;
  while (stream->Read(&note)) {
    for (const RouteNote& n : received_notes) {
      if (n.location().latitude() == note.location().latitude() &&
          n.location().longitude() == note.location().longitude()) {
        stream->Write(n);
      }
    }
    received_notes.push_back(note);
  }

  return Status::OK;
}
```

2) Modification in the underlying C wrapper:

In this task, it will be required to implement a totally new cpp file named Streaming.cpp and Streaming.h to define three separate functions for the streaming techniques (ListFeatures(), RecordRoute() and RouteChat()). The streaming will be avilable to any channel. It will be also avilable for authenticated and encrypted channels as described in the previous sections.

3) Regarding the R code of client.r and server.r nothing will be changed to the point as the start_server and grpc_client function used in the Rcpp function was already modified in the cpp file.

4.2. Testing and Validation on various test case scenarios

It is necessary to test my code or different test case scenarios for various connections and check if the output is as desired. If the code doesn't work as expected, then I need to go back to the drawing board and debug it and write new code if required. plan on giving this task sufficient time, atleast 2 weeks as we need our code to output correct results under different conditions for connections.

I can use the examples provided previusly in the package to check my code in the starting stages. Latter on, at more advanced stages, I will be able to provide and even implement more complex examples invloving various scenarios to test my code efficiently.

Github Link to Examples → https://github.com/nfultz/grpc/tree/master/demo

Main examples which will be used in the initial stages to test my code:

1) helloclient.R
2) helloserver.R

4.3. Make gRPC package ready to be avilable on CRAN

After the code has been tested , I'll then work on getting the package ready to be pushed to CRAN. CRAN provides discoverability, ease of installation and a stamp of authenticity. Hence, the packages need to be well tested and documented before being submitted to the CRAN moderators.

1) **Documentation of each and every function**

   Initially, I plan to document all newly implemented functions normally. Then, I will use the roxygen2 package for ease documentation. I will follow the following steps to successfully accomplish this task:

   1) I will use roxygen2 package to create documentation blocks.
   2) I will run the command devtools::document to:
      - Convert all the created blocks to separate Rd files.
      - Update the NAMESPACE file.

2) **Writing tests for the code**

In order to verify that the newly implemented functions in the package do what they're intended to do, I will test them on edge cases and error conditions to make sure that they are handled appropriately. I will be using the testthat package in addition to R CMD CHECK for proper testing. It is possible that some tests may take over a minute. These tests may be aborted because CRAN runs the tests on all major platforms and has a ceiling for test runtimes. However, this can be handled using the command line skip_on_cran().

3) **Closing the open issues**

Currently there are 5 open issues.
I already provided approaches in the Medium and Hard tests to close the following issues:

1) #28 (Support configurable client deadlines in call())
   Github Link → www.github.com/Shehab7osny/GSoc2020-GRPC-Tests/tree/master/Medium
2) #21 (Installing on windows)
   Github Link →

I will try to close the remaining issues as well but giving priority to implementing the main required functions.

4) **Make sure the package passes all build tests :**
   1) TravisCI will be used for build tests.
   2) covr and codecov will be used for code coverage.

5) **Cleanup of code before pushing it to Github**

After performing all the tests mentioned previously, the code will be cleaned before being pushed to Github. I will do the following to successfully accomplish this task:

1) Removing all the unnecessary comments.
2) Removing the roxygen blocks.
3) Adding descriptive comments for the functions.

This will make it easier for the newer developers to understand the source of the package. This will be done right before pushing the code and will take no time at all.

4.4. Creating the Reference Manual

The manual is created by default when R CMD check is run. The DESCRIPTION file is used to create the PDF reference manual. R internally creates a Latex document but deletes it after the PDF is created. If the manual needs to be edited, --no-clean will be added to the command line, which will provide us with the tex file which can be then edited.

4.5. Development Environment

Following are the tools that I'll be using throughout the coding period. I've tried to cover everything that I might require for the

- Operating System:
  Linux (Windows will be avilable if needed)

- Coding:
  RStudio (for R)
  Visual Studio (for C++)

- Version Control:
  Git + Github

- Code Testing/Coverage:
  Rcpp Package, testthat package, Travis-CI; covr package, codecov.io

- Documentation:
  roxygen2 package

5. Timeline

Pre Coding Phase (April 1 – May 31)

- **From April 1 to May 4**

  Although this is not a part of GSoC program, I will exploit this period to self study and get ready in case I got accepted to start coding without any latency. I will be giving minimum of 25 hours a week for this project during this period to accomplish these tasks:

  - Familiarize myself more with the gRPC package.
  - Get more familiar with the C++ Package of gRPC to make use of it.
  - Trying to close the rest of opened issues.

- **From May 4 to May 31**

  - Discuss the strategies and timeline with the mentors for any suggested modification.
  - Remain active regulary to efficiently discuss those startgies.
  - Finalize the implementation strategy of functions' implementation in addition to any modifications suggested by the mentors.

Phase 1 (June 1 – June 29)

Note that I will be involved with my university Major examination from June 1 to June 15. So, I will interact less. However, I can give 10 hrs per week to my project/community during this time period. I assigned small tasks in this period.

- **From June 1 to June 8**

  - Start implementing the TLS_Secure_Channel() function in the underlying C code.
  - Prepare some test cases in C++ to test this function on a regular basis until it outputs correct results.
  - Simultaneously Start documention for this function.

- **From June 8 to June 15**

  - Continue implementation of the TLS_Secure_Channel() function.
  - Test the functionallity regularly. If errors encountered, correct them.
  - Get the tests done for the TLS_Secure_Channel() function.

- **From June 15 to June 22**

  - Test the TLS_Secure_Channel() function and match the output results with the expected ones.
  - If errors encountered, correct them.
  - Write examples for the function while testing it.
  - Link the cpp function to the R file using Rcpp Package.
  - Simultaneously document this function along with the examples.

- **From June 22 to June 29**

  - Test the R file using the examples provided in the demo.
  - Make sure the package passes all build tests on all platforms.
  - Final check with R CMD check.
  - Finalize the documentation for this function.

Phase 1 Evaluation (June 29 - July 3)

- **From June 29 to July 3**

  - Discuss the drawbacks of Phase 1 with the mentor and ask for any improvements.
  - Ask the mentor for any additional info required for Phase 2.
  - Gain any missing information related to Phase 2

Phase 2 (July 3 - July 27)

- **From June 3 to July 6**

According to the technical plan, since the steps of implementing Authenticate_Channel() function are similar to TLS_Secure_Channel() function. It can be accomplised in smaller duration due to the gained experience.

  - Start implementing the Authenticate_Channel() function in the underlying C code.
  - Prepare some test cases in C++ to test this function on a regular basis until it outputs correct results.
  - Simultaneously Start documention for this function.

- **From July 6 to July 13**

  - Continue implementation of the Authenticate_Channel() function.
  - Test the functionallity regularly. If errors encountered, correct them.
  - Get the tests done for the Authenticate_Channel() function.

- **From July 13 to July 20**

  - Test the Authenticate_Channel() function and match the output results with the expected ones.
  - If errors encountered, correct them.
  - Write examples for the function while testing it.
  - Link the cpp function to the R file using Rcpp Package.
  - Simultaneously start implementing the Streaming() function in the underlying C code.

- **From July 20 to July 27**

  - Test the R file using the examples provided in the demo.
  - Make sure the package passes all build tests on all platforms.
  - Final check with R CMD check.
  - Finalize the documentation for Authenticate_Channel() function.
  - Simultaneously, for Streaming() function, prepare some test cases in C++ to test this function on a regular basis until it outputs correct results.

Phase 2 Evaluation (July 27 - July 31)

- **From July 27 to July 31**

  - Discuss the drawbacks of Phase 2 with the mentor and ask for any improvements.
  - Ask the mentor for any additional info required for the final Phase.
  - Gain any missing information related to the final Phase.

Final Phase (August 1 - August 8)

- **From August 1 to August 8**

  - Start documention for the Streaming() function.
  - Continue implementation of the Streaming() function.
  - Test the functionallity regularly. If errors encountered, correct them.
  - Get the tests done for the Streaming() function.
  - Test the function and match the output results with the expected one.
  - If errors encountered, correct them.

- **From August 8 to August 15**

  - Write examples for the function while testing it.
  - Link the cpp function to the R file using Rcpp Package.
  - Test the R file using the examples provided in the demo.
  - Make sure the package passes all build tests on all platforms.
  - Final check with R CMD check.
  - Finalize the documentation for Streaming() function.

- **From August 15 to August 24**

  - This is a A buffer of two weeks kept for any unpredictable delay.
  - Finalize the documentation, Fix issues and merge the code into gRPC's repositories.
  - Submission of project report
  - I will love to display the project work to the community members and get the review and feedback for changes if any.
  - Work on a blog post about the tasks accomplished during the coding period.

6. Tests Solution

In this section I will describe briefly my approaches to solve the tests provided by the mentors (Easy – Medium – Hard). I solved each of these tests on my own and I did some research to be able to come for a solution for the Hard Test.

6.1. Easy Test

(Github link: www.github.com/Shehab7osny/GSoc2020-GRPC-Tests/tree/master/Easy)

Requirement:
Write an Rcpp Function.

- **Function Arguments:**
  - **Integer ➡ n**
  - **R Function ➡ Callback**
- **The Rcpp Function should do the following:**
  - **Draw n Samples from the Normal Distribution.**
  - **Execute the Callback function on each sample that is Greater Than Zero.**

Solution:

1) Load the Rcpp library using the following command line:
   To provides R functions as well as C++ classes which offer a seamless integration of R and C++.

```
> library(Rcpp)
```

2) In the Script, Write the C++ function inside the function cppFunction():

```
cppFunction('
        void Samples_Norm(int n, Function callback)
        {
            NumericVector Norm_Dist_Values = rnorm(n);

            for (int i = 0; i < n; i++)
           {
              if (Norm_Dist_Values[i] > 0)
              {
                 callback(Norm_Dist_Values[i]);
              }
           }
           }
          ')
```

C++ Code Explanation:

- `NumericVector`
  Datatype to define a Vector [`Norm_Dist_Values`] of Numeric Values.

- `rnorm(n)`
  To generate n different Normal Distribution Values.

3) Call the Rcpp function the Console:

```
> Samples_Norm(50, print)
```

4) Output in the console:

```
[1] 2.432729
[1] 0.5920912
[1] 0.4066282
[1] 0.4409
[1] 2.235376
[1] 0.5993602
[1] 1.274233
[1] 0.08148341
[1] 0.3378953
[1] 0.5661375
[1] 1.72677
[1] 1.588315
[1] 0.3695074
[1] 0.4386581
[1] 0.7609176
[1] 0.06428198
[1] 2.298479
[1] 1.488323
```

6.2. Medium Test

(Github link: www.github.com/Shehab7osny/GSoc2020-GRPC-Tests/tree/master/Medium)

Requirement:
Modify the grpc client timeout to take it's value from options().

Solution:

1) Open the File needed to be modified:
   Link to Github used → https://github.com/nfultz/grpc/issues/28
   File to be modified → src/client.cpp

   Snippet of Code to be modified:

```
...
RawVector fetch(CharacterVector server, CharacterVector method,
RawVector requestArg, CharacterVector metadata)
{

//Timeout to be modified
gpr_timespec deadline = five_seconds_from_now();

//setup
grpc_call *c;
...
```

2) Timeout function for a Configurable Time:
Link to Github used:
https://github.com/grpc/grpc

File to be modified:
grpc/test/core/end2end/tests/authority_not_supported.cc

Required Function:

```
...
static gpr_timespec n_seconds_from_now(int n) {
    return grpc_timeout_seconds_to_deadline(n);
}
...
```

Required Function: `n_seconds_from_now(int n)`
This function uses the pre-defined function [`grpc_timeout_seconds_to_deadline(n)`] to configure the Client Timeout so that it can be set by the user to any variable n.

3) Modify the Original Code:

My approach to modify the fetch() function is to add an additional argument n which represents the number of seconds needed for the Client to Timeout. I don't know exactly where the options() is, so that would be my approach until it is clarified.

Modified Code:

```
...
RawVector fetch(CharacterVector server, CharacterVector method,
RawVector requestArg, CharacterVector metadata, int n)
{

//Timeout after modification
gpr_timespec deadline = n_seconds_from_now(n);

//setup
grpc_call *c;
...
```

I assume that the header files are correctly included and that the function [`n_seconds_from_now(n)`] is correctly prototyped in the header file and defined in the cpp file. However, if it is not, then I may define the function in the client.cpp file itself for proper use.

This will allow the user to set a Configurable Time in seconds for the Client to Timeout.

4) Modify the R Code of the Client:

Snippet of Code to be modified:

```
...
grpc_client <- function(impl, channel) {

client_functions <- lapply(impl, function(fn, n)
{
  RequestDescriptor <- P(fn[["RequestType"]]$proto)
  ResponseDescriptor <- P(fn[["ResponseType"]]$proto)
  list(
          call = function(x, metadata=character(0))
          read(ResponseDescriptor, fetch(channel, fn$name,
          serialize(x, NULL), metadata, n)),
...
```

The new argument representing the Configurable Time is added to the function arguments in the R File.

6.3. Hard Test

(Github link: www.github.com/Shehab7osny/GSoc2020-GRPC-Tests/tree/master/Hard)

Requirement:
Build and check the grpc package on Windows

Solution:

1) Use the windows prebuild binary based on mingw using MSYS2.

2) Installed MSYS2 and gRPC with

```
pacman -S mingw-w64-x86_64-grpc
```

3) Created Makevars.win containing:

```
PKG_CPPFLAGS=-IC:/msys64/mingw64/include -D_WIN32_WINNT=0x600
PKG_LIBS=-LC:/msys64/mingw64/lib -lgrpc -lgpr
```

4) Use the following command line to install the package locally:

```
install_local("Path of the Package in zip format",
INSTALL_opts = c("--no-multiarch", "--no-test-load"))
```

5) Download the dll file in the Github link and place it in the following path . . . R\R-3.6.3\library\grpc\libs\x64.

7. Additional Commitments

8. Why this Project?

9. Why me?